



7 LangChain Complex Topics

Interviewers Love ❤️ but
Students Skip 🎮

LCEL

Runnables

Memory

Vector Stores

Tools

Agents

Prompts



Naved Khan
Gen AI Engineer

+ Follow

LCEL - LangChain Expression Language

LCEL is a declarative syntax for composing LangChain components using the **pipe operator (|)**. It provides built-in support for streaming, async, batch processing, and parallel execution.

Pipe Operator (|)

Chains components sequentially where output of one becomes input to the next.

Built-in Methods

invoke() – single execution | **stream()** – token streaming | **batch()** – multiple inputs | **ainvoke()** – async

```
chain = prompt | model | parser
result = chain.invoke({"topic": "AI"})
```

🔗 INTERVIEW Q&A

Q: Why use LCEL over legacy chains?

A: LCEL provides native streaming, async support, batch processing, and parallel execution without additional configuration.



Naved Khan
Gen AI Engineer

Runnables - The Building Blocks

Runnables are the base interface in LangChain. All composable components implement the Runnable protocol, enabling uniform composition via the pipe operator.

RunnableLambda

Wraps any Python function into a Runnable, enabling custom logic within LCEL chains.

RunnablePassthrough

Passes input unchanged. Use `.assign()` to add computed values while preserving original input.

RunnableParallel

Executes multiple Runnables concurrently. Syntax: `{"key1": chain1, "key2": chain2}`

INTERVIEW Q&A

Q: How to run multiple chains in parallel?

A: Use RunnableParallel or dictionary syntax. Both execute concurrently.



Naved Khan

Gen AI Engineer

Runnables - Advanced Types

RunnableBranch enables conditional routing in LCEL chains. It evaluates conditions sequentially and routes to the first matching branch.

RunnableBranch Syntax

Takes (condition, runnable) tuples and a default runnable.

Common Methods

`invoke()` | `ainvoke()` | `stream()` | `batch()`

```
branch = RunnableBranch(  
    (cond_1, chain_1),  
    (cond_2, chain_2),  
    default_chain)
```

INTERVIEW Q&A

Q: How to implement conditional logic in LCEL?

A: Use `RunnableBranch` with (condition, runnable) pairs. Routes to first match or default.



Naved Khan

Gen AI Engineer

Memory Management

Memory enables state persistence across interactions. LangGraph provides checkpoints for short-term and stores for long-term persistence.

Short-term Memory (Checkpoint)

Thread-scoped persistence using **InMemorySaver**. Maintains conversation history within a session.

Long-term Memory (Store)

Cross-thread persistence using **InMemoryStore**. Stores user preferences across sessions.

Thread ID

Unique identifier for conversation sessions. Config: `{"thread_id": "123"}`



Vector Stores & Retrieval

Vector Stores are databases for storing and querying vector embeddings. They enable semantic search by finding documents with similar meaning.

Core Operations

from_documents() – create | **similarity_search()** – query |
as_retriever() – LCEL compatible

Popular Implementations

Chroma – local | **FAISS** – in-memory | **Pinecone** – cloud | **Qdrant** – self-hosted

```
db = Chroma.from_documents(docs, embeddings)
results = db.similarity_search(query)
retriever = db.as_retriever()
```

🎯 INTERVIEW Q&A

Q: `similarity_search()` vs `as_retriever()`?

A: `similarity_search()` returns docs directly. `as_retriever()` returns a Runnable for LCEL chains.



Naved Khan
Gen AI Engineer

Custom Tools with @tool

Tools extend LLM capabilities by enabling interaction with external systems. The **@tool** decorator converts Python functions into tool objects.

@tool Decorator

Converts a function to a Tool. The docstring becomes the tool description for LLM.

Required Components

Type hints – define input schema | **Docstring** – describes tool purpose

```
@tool
def get_weather(city: str) → str:
    """Get weather for a city."""
    return weather_api.get(city)
```

🎯 INTERVIEW Q&A

Q: How does LLM decide which tool to use?

A: LLM reads tool names and docstrings to determine which tool matches the request.



Naved Khan
Gen AI Engineer

Agents with `create_agent()`

Agents are autonomous systems combining LLMs with tools. They use the **ReAct pattern** (Reason + Act) to iteratively decide actions.

Agent Loop

Reason → **Act** (call tool) → **Observe** → **Repeat** until complete

`create_agent()`

LangChain v1 standard. Built on LangGraph. Replaces deprecated `initialize_agent()`.

```
from langchain.agents import create_agent
agent = create_agent(
    model="gpt-4", tools=[search])
```

⚠️ TRAP QUESTION

Q: How to create agents in LangChain?

A: Use `create_agent()` (v1). `initialize_agent()` is deprecated. Built on LangGraph.



Prompt Templates

Prompt Templates are reusable prompt structures with variable placeholders. They ensure consistent formatting using `{variable}` syntax.

`from_template()`

Creates single-message prompts. Use for simple prompts without role specification.

`from_messages()`

Creates multi-message prompts with role tuples: (role, content). Supports system, human, AI.

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are helpful."),
    ("human", "{question}")])
```

🎯 INTERVIEW Q&A

Q: `from_template()` vs `from_messages()`?

A: `from_template()` for simple prompts. `from_messages()` for system instructions + roles.



Naved Khan
Gen AI Engineer

Want more content like this?



Naved Khan

Gen AI Engineer

*Let's connect! Follow for more
simple tech explanations.*



Like



Comment



Share



Save